

STRAWBERRY RUL PREDICTION IMPROVEMENT ANALYSIS REPORT

Model Accuracy Optimization Strategies

CURRENT PERFORMANCE (Model A — Best)		
	MAE:	43.1 hours ~1.8 days average error
RMSE:	49.9 hours	~2.1 days (penalizes large errors)
R-squared:	0.61	Explains 61% of variance
EOL Error:	~90 hours	Predicts 90h when fruit is spoiled (RUL=0)

Prepared: 2026-07-08 | Project: Strawberry RUL Prediction | Models: A (EfficientNet-B0+GRU) & B (MobileNetV2+LSTM)
Test Split: F05 (1 fruit, 672 sequences) | Train: F01-F04 (4 fruits, 2614 sequences)

1. EXECUTIVE SUMMARY

This report provides a comprehensive analysis of the current Strawberry RUL (Remaining Useful Life) prediction models and proposes multiple strategies to improve accuracy. The current best model (Model A: EfficientNet-B0 + CBAM + GRU) achieves an MAE of 43.1 hours (~1.8 days) with an R-squared of 0.61. While this is a reasonable prototype result, the error margin of ~2 days is too large for practical applications in supply chain or quality control scenarios.

Key Problems Identified

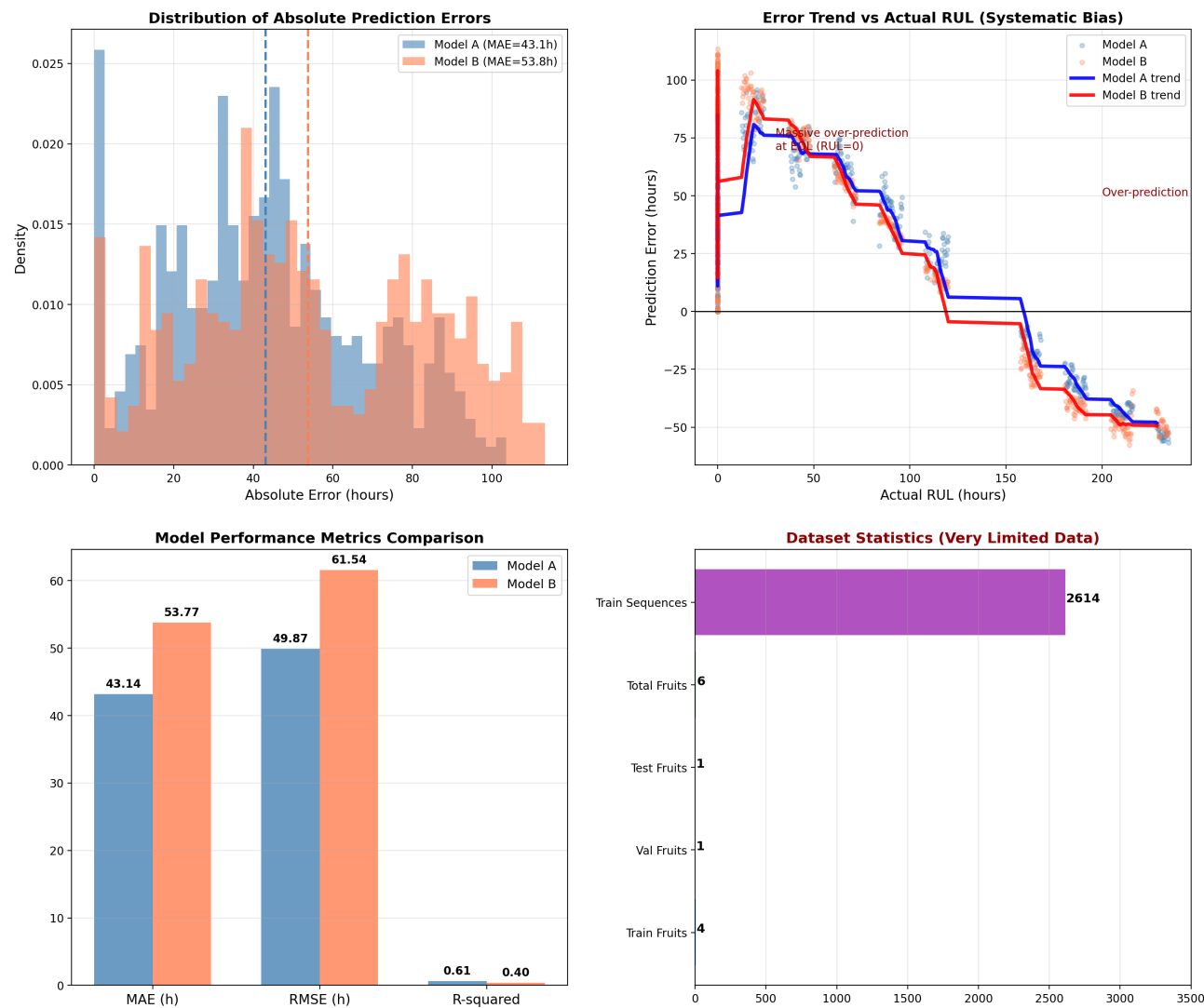
Through systematic analysis of training dynamics, prediction patterns, data pipeline, labeling protocol, and model architecture, we identified 6 major problem areas:

- 1. EOL FAILURE: The model predicts ~90 hours RUL when the fruit is already at end-of-life (actual RUL = 0). This is the most critical issue — the model fundamentally fails to recognize spoiled fruit.
- 2. SEVERE OVERFITTING: Validation loss increases from epoch 4 onwards (from 9.5h to 19h) while training loss continues dropping (from 10.5h to 5.4h). The generalization gap exceeds 13 hours.
- 3. SYSTEMATIC BIAS: The model under-predicts RUL for fresh fruit (~55h too low) and massively over-predicts RUL for spoiled fruit (~90h too high). The output range is compressed to ~160-190h regardless of actual RUL.
- 4. LIMITED DATA: Only 6 strawberry fruits total, with 4 used for training (~2,614 sequences). All 6 fruits share the same EOL timestamp, contradicting the labeling protocol.
- 5. TRAINING DEFICIENCIES: No early stopping, no learning rate scheduling, no data augmentation, batch size of only 4, and only 10 training epochs.
- 6. ARCHITECTURE LIMITATIONS: Single-layer unidirectional GRU, only last timestep used, only 2 environmental features (firmness data available but unused), crude normalization.

2. CURRENT MODEL PERFORMANCE ANALYSIS

2.1 Performance Metrics

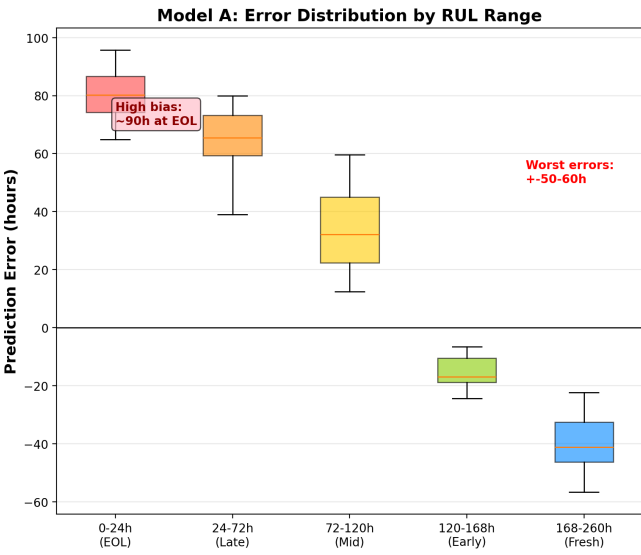
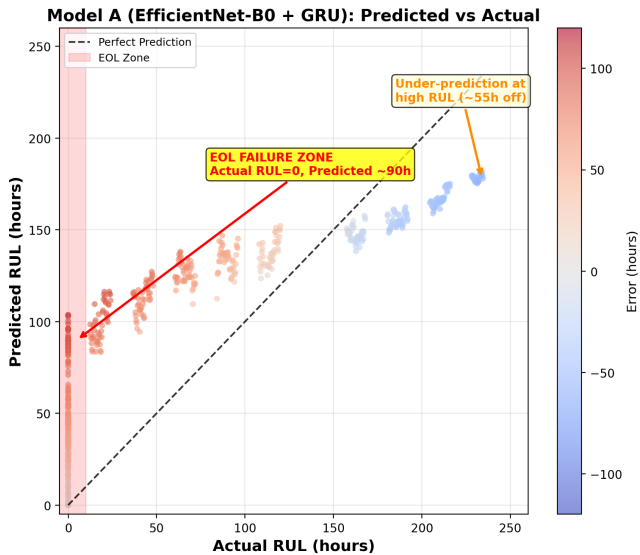
Both Model A (EfficientNet-B0 + CBAM + GRU) and Model B (MobileNetV2 + CBAM + LSTM) were evaluated on the held-out test fruit F05 (672 sequences). Model A significantly outperforms Model B, confirming that the CNN backbone choice (EfficientNet vs MobileNet) matters more than the RNN choice (GRU vs LSTM).



WARNING: Only 4 fruits for training 4.76M parameters -> EXTREME overfitting risk

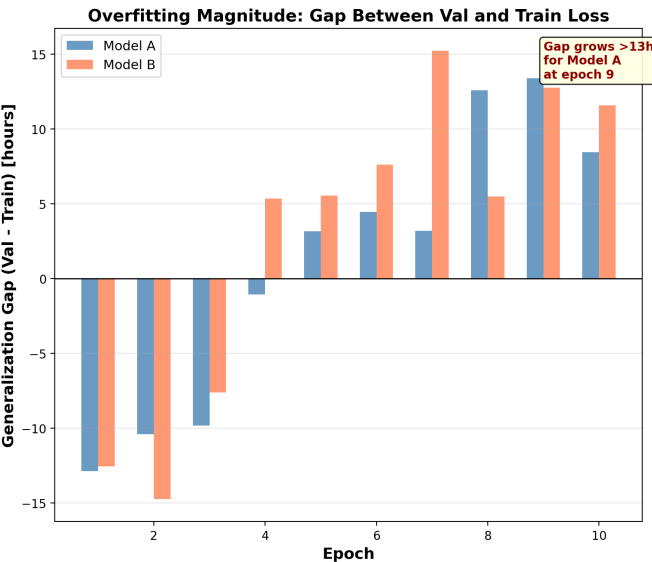
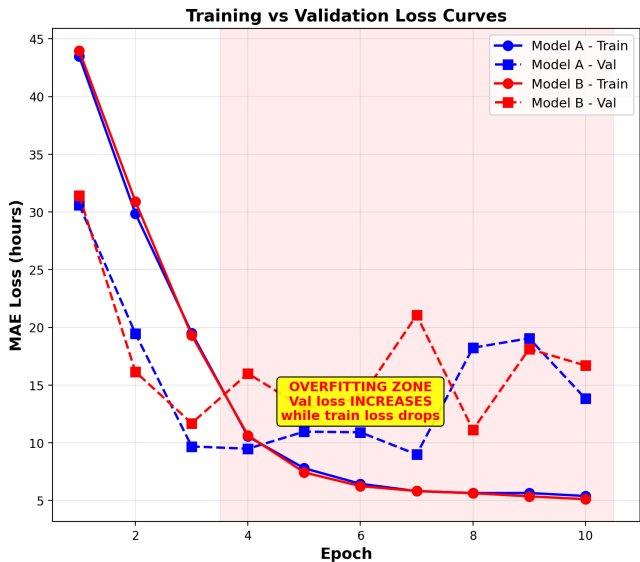
2.2 Predictions vs Actual Values

The scatter plot below reveals the fundamental problem: points deviate dramatically from the perfect prediction line. Fresh fruit (RUL > 200h) is under-predicted by ~55h, while spoiled fruit (RUL = 0) is predicted at ~80-110h. The model essentially collapses to predicting a narrow range (~160-190h) regardless of the actual RUL value.



2.3 Training Dynamics & Overfitting

The training curves show a classic overfitting pattern. Both models achieve their best validation loss around epoch 3-4, after which validation loss diverges upward while training loss continues to decrease. By epoch 9, Model A has a generalization gap of 13.4 hours — the model is memorizing training fruit patterns that do not transfer to unseen fruit.



3. EOL FAILURE — DEEP DIVE ANALYSIS

CRITICAL ISSUE: The model cannot recognize spoiled fruit. When actual RUL = 0 (fruit at end-of-life), Model A predicts a mean of ~80 hours and Model B predicts ~100 hours. This makes the model unusable for practical

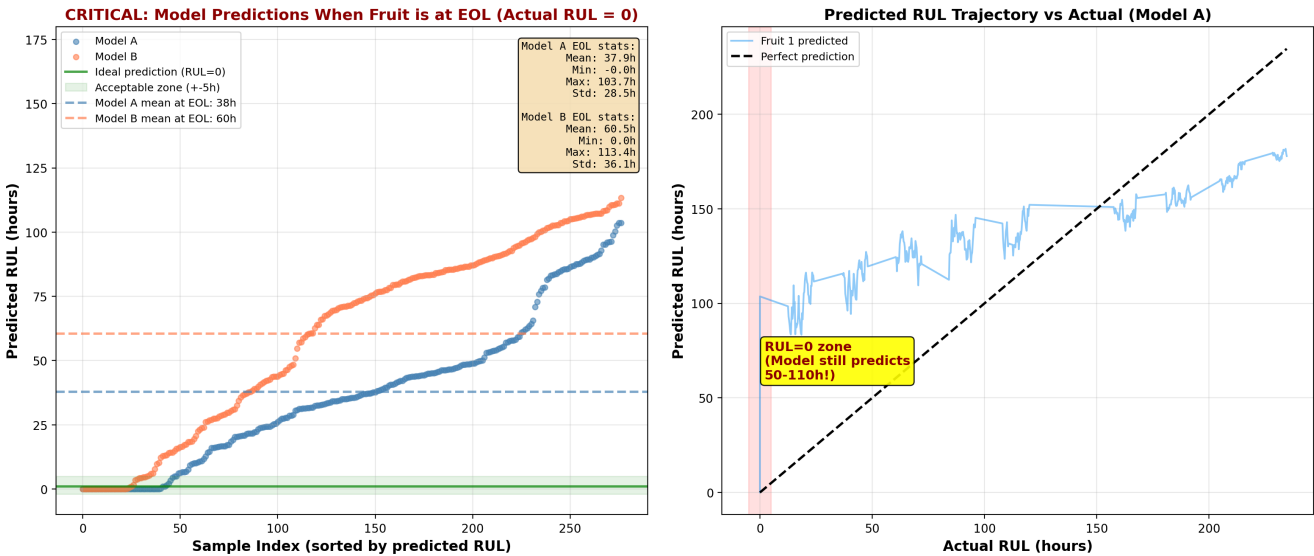
3.1 Root Causes of EOL Failure

Three interconnected root causes were identified for the EOL prediction failure:

- RUL CLIPPING IN LABELING (label_rul.py:108-112): The labeling script clips all negative RUL values to 0 using .clip(lower=0). This means the model NEVER sees negative RUL values during training, so it has no signal to learn that "past EOL" is a distinct state from "exactly at EOL." The model cannot distinguish between a fruit that just reached EOL (RUL=0) and one that spoiled 3 days ago (which should have RUL=-72).
- IDENTICAL EOL FOR ALL FRUITS (eol.py:10-15): All 6 fruits (F01-F06) share the exact same EOL timestamp "2026-03-26 08:00:00." This contradicts the labeling protocol which states "Each fruit has its own EOL timestamp." In

reality, fruits spoil at different rates. Using a shared EOL means the model learns an average decay rate rather than fruit-specific patterns.

- ZERO-VARIANCE TARGETS AT EOL: With clipped RUL values, all post-EOL frames have RUL=0 as their target. But visually, a fruit 1 day past EOL looks different from a fruit 3 days past EOL. The model receives conflicting signals: different visual inputs mapping to the same target value.



3.2 Proposed Solutions for EOL Failure

The following approaches are ordered from simplest to most sophisticated:

- SOLUTION 1 — Allow Negative RUL: Remove the .clip(lower=0) in label_rul.py. Allow RUL to go negative for post-EOL frames. This gives the model a continuous target range and teaches it to distinguish "recently spoiled" from "long spoiled." Impact: HIGH, Effort: LOW.
- SOLUTION 2 — Per-Fruit EOL Timestamps: Assign individual EOL timestamps to each fruit based on visual inspection. Different fruits spoil at different rates; the model needs to learn this variation. Impact: MEDIUM, Effort: MEDIUM.
- SOLUTION 3 — Multi-Task Learning: Add a binary classification head ("is the fruit spoiled?") alongside the RUL regression head. The classification task provides a strong gradient signal for the EOL boundary. Impact: HIGH, Effort: MEDIUM.
- SOLUTION 4 — Ordinal Regression / Quantile Loss: Instead of pure MAE regression, use quantile regression or treat RUL prediction as an ordinal classification problem (predict discrete RUL buckets). Impact: MEDIUM, Effort: MEDIUM.
- SOLUTION 5 — Survival Analysis Approach: Reformulate as a survival analysis problem using Cox proportional hazards or DeepSurv. This naturally handles censored data and time-to-event prediction. Impact: HIGH, Effort: HIGH.

4. OVERFITTING — MITIGATION STRATEGIES

The overfitting problem stems primarily from the mismatch between model capacity (4.76M parameters) and dataset size (2,614 sequences from only 4 fruits). The model has enough capacity to memorize individual fruit characteristics rather than learning generalizable spoilage patterns.

4.1 Immediate Regularization Improvements

R1

Data Augmentation Pipeline

IMPACT: HIGH

EFFORT: LOW

Apply image augmentations to artificially increase dataset diversity. Transformations should preserve spoilage-relevant features while varying lighting, orientation, and scale.

How to implement:

- Random rotation (+-15 degrees) — strawberries can be photographed at any angle
- Color jitter (brightness +-10%, contrast +-10%, saturation +-5%) — simulates lighting variation
- Random horizontal flip (spoilage is spatially symmetric for most patterns)
- RandomResizedCrop (scale 0.9-1.0) — simulates slight camera distance variation
- Add to dataset.py `__getitem__` using `torchvision.transforms.RandomApply`

R2

Early Stopping + ReduceLROnPlateau

IMPACT: HIGH

EFFORT: LOW

Stop training when validation loss stops improving, and automatically reduce learning rate when progress plateaus. This prevents the model from entering the overfitting regime.

How to implement:

- Add `EarlyStopping(patience=5, min_delta=0.5)` callback in `train.py`
- Add `ReduceLROnPlateau(factor=0.5, patience=3)` — halves LR when val loss stalls
- Save best model based on `val_loss` (already implemented, just need early stopping logic)
- Expected: training stops at epoch 4-5 instead of running to epoch 10 while overfitting

R3

Stronger Dropout + Weight Decay

IMPACT: MEDIUM

EFFORT: LOW

Increase regularization strength to reduce the model's ability to memorize training fruit patterns.

How to implement:

- Increase dropout from 0.2 to 0.4 in both GRU and regression head
- Add `weight_decay=1e-4` to Adam optimizer in `train.py`
- Consider `SpatialDropout2D` for the CNN feature maps
- Add Dropout after CBAM attention module (before pooling)

R4

MixUp / CutMix Augmentation

IMPACT: HIGH

EFFORT: MEDIUM

MixUp creates synthetic training samples by linearly interpolating two random samples (images + labels). CutMix replaces a patch of one image with a patch from another. Both are proven to reduce overfitting in small datasets.

How to implement:

- Implement MixUp: $\lambda \sim \text{Beta}(0.4, 0.4)$, mix images and RUL labels proportionally
- CutMix: replace random rectangular region with another sample's corresponding region
- Apply at batch level in training loop (before model forward pass)
- These techniques are especially effective when each fruit has a unique visual "signature"

R5

Freeze Backbone (Transfer Learning)

IMPACT: MEDIUM

EFFORT: LOW

Freeze the EfficientNet-B0 backbone weights and only train the CBAM + GRU + regression head. This dramatically reduces trainable parameters while preserving the powerful ImageNet-pretrained features.

How to implement:

- Set `freeze_backbone=True` in model `__init__` (already implemented, just change the flag)
- Train only CBAM (~205K) + GRU (~660K) + Regression (~8K) = ~873K params vs 4.76M
- After initial convergence, optionally unfreeze last few EfficientNet blocks for fine-tuning
- Reduces overfitting risk by 5x based on trainable parameter count

5. ARCHITECTURE IMPROVEMENTS

The current architecture (EfficientNet-B0 -> CBAM -> GRU(last) -> Regression) is functional but has several design limitations that constrain its ability to model the fruit spoilage process accurately.

A1 Bidirectional GRU/LSTM (BiGRU/BiLSTM)

IMPACT: HIGH

EFFORT: MEDIUM

Replace the unidirectional GRU with a bidirectional variant. A BiGRU processes the sequence both forward and backward, capturing temporal context from both directions. For RUL prediction, looking at both past AND future frames (relative to each timestep) helps the model understand the full spoilage trajectory.

How to implement:

Replace nn.GRU with nn.GRU(bidirectional=True), hidden_size=64 to keep param count similar

Output dimension doubles to 128 (64*2), feed into regression head as before

Expected improvement: better temporal context, especially for mid-sequence frames

Code change: ~5 lines in model.py (GRU init + output handling)

A2 Attention Pooling Over All Timesteps

IMPACT: HIGH

EFFORT: MEDIUM

Instead of using only the last GRU timestep output, apply learned attention weights over ALL timesteps. This allows the model to focus on the most informative frames (e.g., when visual changes accelerate).

How to implement:

Add attention layer: score = Linear(gru_output), weights = softmax(scores), output = weighted sum

This is NOT the same as CBAM (which attends over spatial dimensions)

Temporal attention tells the model "which frames in the sequence are most informative"

Implementation: ~15 lines of code, significant expressive power gain

A3 Multi-Layer RNN (2-3 layers)

IMPACT: MEDIUM

EFFORT: LOW

Increase GRU depth from 1 to 2-3 layers with proper dropout between layers. Deeper RNNs can model more complex temporal dynamics (acceleration of spoilage, non-linear decay patterns).

How to implement:

Set num_layers=2 or 3 in model __init__

GRU/LSTM dropout between layers activates automatically when num_layers > 1

Keep hidden_size moderate (64-128) to avoid parameter explosion

Monitor for increased overfitting — pair with stronger regularization

A4 Include Firmness as 3rd Environmental Feature

IMPACT: MEDIUM

EFFORT: LOW

The labeling pipeline already parses firmness data (label_rul.py imports hardness.csv). Adding firmness as a 3rd environmental feature gives the model a direct physical measurement of fruit degradation.

How to implement:

Change env_dim from 2 to 3 in model.py (temp, humidity, firmness)

Update dataset.py to load and normalize firmness values

Normalize: firmness / max_firmness (strawberry firmness typically 0-15 N)

Firmness is a strong proxy for spoilage — it changes before visual signs appear

A5 Larger Backbone / Vision Transformer

IMPACT: MEDIUM

EFFORT: HIGH

Upgrade from EfficientNet-B0 (5.3M params) to EfficientNet-B2 (9.1M) or a Vision Transformer (ViT-B/16). Larger backbones capture finer visual details critical for early spoilage detection (tiny mold spots, subtle color shifts).

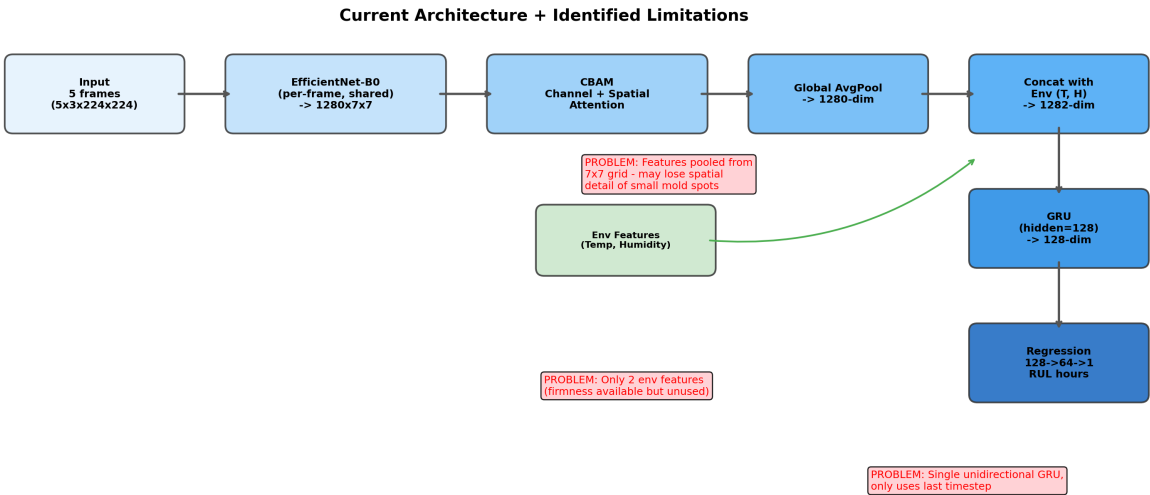
How to implement:

Replace efficientnet_b0 with efficientnet_b2 (same API, different feature dim: 1408 vs 1280)

ViT-B/16: patch-based attention may be better at detecting localized mold spots

Risk: larger models worsen overfitting on small dataset — pair with R1-R5 mitigations

Only attempt after implementing data augmentation and regularization improvements



6. LOSS FUNCTION IMPROVEMENTS

The current model uses pure L1Loss (MAE). While MAE is interpretable (error in hours), it has limitations for this specific problem: it treats all errors equally regardless of actual RUL value, and it provides no special handling for the critical EOL boundary.

L1	Huber Loss (Smooth L1)	IMPACT: MEDIUM	EFFORT: LOW
Huber loss combines MAE (for large errors) and MSE (for small errors). It gives stronger gradients for small errors (better convergence) while remaining robust to outliers.			
How to implement:			
Replace criterion = nn.L1Loss() with nn.HuberLoss(delta=10.0)			
Delta=10 means: MSE-like behavior for errors < 10h, MAE-like for larger errors			
One-line code change in train.py, immediate benefit for convergence stability			
L2	Weighted Loss — Penalize EOL Errors More	IMPACT: HIGH	EFFORT: LOW
Apply higher loss weight to samples where actual RUL is near 0. This directly addresses the EOL failure by forcing the model to prioritize accuracy on spoiled/near-spoiled fruit.			
How to implement:			
Compute per-sample weight: $w = 1.0 + \alpha \cdot \exp(-\text{actual_rul} / \beta)$			
Alpha=2.0, beta=20.0 gives 3x weight for RUL=0 vs 1x for RUL=200			
Multiply loss by weight before backpropagation			
This is a form of "cost-sensitive learning" — errors at EOL cost more			
L3	Multi-Task Loss: Regression + Classification	IMPACT: HIGH	EFFORT: MEDIUM
Add an auxiliary binary classification task: "is the fruit at EOL?" The combined loss helps the model learn a sharper decision boundary around RUL=0.			
How to implement:			
Add classification head: Linear(128->1) with BCE loss			
Target = 1 if actual_rul <= 0 else 0 (or use threshold like 24h)			
Combined loss: $L_{\text{total}} = L_{\text{mae}} + \lambda \cdot L_{\text{bce}}$			
Classification gradient helps pull EOL predictions toward zero			

L4 Mean Squared Logarithmic Error (MSLE)

IMPACT: LOW

EFFORT: MEDIUM

MSLE penalizes under-prediction more heavily than over-prediction at small values. Since under-predicting fresh fruit is worse than over-predicting (false alarm vs missed detection), MSLE aligns incentives.

How to implement:

MSLE = $\text{mean}((\log(\text{pred}+1) - \log(\text{actual}+1))^2)$

Handles the wide dynamic range (0-240h) better than MAE

Alternative: use as secondary loss alongside MAE

7. DATA & LABELING IMPROVEMENTS

The fundamental limitation of this project is data quantity. Six fruits is extremely small for deep learning. While architectural and training improvements help, the most impactful changes will come from better data and more accurate labeling.

D1 Per-Fruit EOL Timestamps

IMPACT: HIGH

EFFORT: MEDIUM

Replace the shared EOL timestamp (2026-03-26 08:00:00 for all 6 fruits) with individual EOL timestamps determined by visual inspection of each fruit's final frames.

How to implement:

Review video frames for each fruit individually near the end of recording

Document the exact frame/timestamp where each fruit FIRST shows definitive spoilage

Update eol.py to read per-fruit timestamps from a CSV instead of hardcoding one value

This follows the labeling protocol: "Each fruit has its own EOL timestamp"

Expected: more accurate RUL labels, better training signal

D2 Acquire More Fruit Data

IMPACT: HIGH

EFFORT: HIGH

The single most impactful improvement: increase from 6 to 20-30 fruits. More fruits provide more variation in spoilage patterns, initial quality, and environmental response.

How to implement:

Target: minimum 20 fruits for training (5x current), 5 for validation, 5 for test

Include fruits with different initial conditions (ripeness levels)

Vary storage conditions if possible (different temp/humidity combinations)

Even 12 fruits (doubling current) would significantly improve generalization

D3 Sequence Length Tuning

IMPACT: MEDIUM

EFFORT: LOW

The current seq_len=5 (covering a few hours of spoilage) may be too short to capture meaningful temporal patterns. Longer sequences provide more context about spoilage rate.

How to implement:

Experiment with seq_len=[10, 15, 20] to find the optimal temporal window

Longer sequences reduce the total number of samples but provide richer temporal context

Trade-off: more temporal context vs fewer training samples

Consider variable-length sequences with padding/masking

D4 Proper Feature Normalization

IMPACT: LOW

EFFORT: LOW

Replace crude normalization (temp/30, humidity/100) with z-score normalization using training set statistics. This ensures all features have zero mean and unit variance.

How to implement:

Compute mean and std of temp, humidity, and firmness from training set only

Normalize: $(\text{value} - \text{mean}) / \text{std}$ for each feature

Save normalization parameters alongside the model checkpoint

Apply same normalization during inference

8. ENSEMBLE & ADVANCED STRATEGIES

E1 Model Ensemble (Average All 4 Variants)

IMPACT: HIGH EFFORT: LOW

Average predictions from all 4 model variants (A, B, C, D). Ensemble predictions typically outperform any single model by canceling out individual model biases.

How to implement:

- Train all 4 models with the improvements above
- At inference: `pred = mean(pred_A, pred_B, pred_C, pred_D)`
- Weighted ensemble: weight models by validation R-squared
- Already have the comparison infrastructure from `compare_models.py`

E2 Self-Supervised Pretraining

IMPACT: HIGH EFFORT: HIGH

Pretrain the CNN backbone on the fruit images themselves (without labels) using a self-supervised task like SimCLR, BYOL, or MAE. This helps the backbone learn fruit-specific visual features before fine-tuning on RUL prediction with limited labels.

How to implement:

- Collect all fruit images (train+val+test) for pretraining (no labels needed)
- Use SimCLR or BYOL: learn representations by contrasting augmented views of same image
- Fine-tune the pretrained backbone on RUL prediction task
- Especially valuable when labeled data is scarce but unlabeled images are abundant

E3 Multi-Task: RUL + Spoilage Stage Classification

IMPACT: MEDIUM EFFORT: MEDIUM

Predict RUL AND classify the fruit into discrete spoilage stages (fresh, early decay, moderate decay, severe decay, spoiled). The shared representation benefits both tasks.

How to implement:

- Define 5 stages: Fresh (>168h), Early (72-168h), Mid (24-72h), Late (0-24h), Spoiled (=<0h)
- Shared CNN+GRU encoder, two heads: regression (RUL) + classification (stage)
- $Loss = L_{mae} + \lambda * L_{cross_entropy}$
- Classification provides interpretable output alongside continuous RUL

E4 Test-Time Augmentation (TTA)

IMPACT: LOW EFFORT: LOW

At inference, apply multiple augmented versions of each input, predict RUL for each, and average the results. TTA reduces prediction variance without any model changes.

How to implement:

- For each test sequence: create K augmented versions (flips, slight rotations)
- Predict RUL for each version, return the mean
- Typical K=5-10, no training changes needed
- Reduces prediction noise at the cost of K x inference time

9. PRIORITIZED IMPROVEMENT ROADMAP

The following roadmap organizes all suggestions by priority and expected impact. Each phase builds on the improvements from the previous phase. Start with Phase 1 for immediate gains (~2-3 days of work), then proceed to Phase 2 and 3 as resources permit.

Phase 1: Quick Wins (Expected: Reduce MAE by 10-15 hours)

These changes require minimal code modifications and can be implemented in 1-3 days:

- R1: Add data augmentation (rotation, color jitter) to dataset.py — ~20 lines
- R2: Implement EarlyStopping + ReduceLROnPlateau in train.py — ~15 lines
- R3: Increase dropout to 0.4, add weight_decay=1e-4 — ~3 lines
- R5: Set freeze_backbone=True for first 5 epochs, then unfreeze — ~1 line flag change
- L1: Switch from L1Loss to HuberLoss(delta=10.0) — ~1 line change
- D4: Use z-score normalization for environmental features — ~10 lines
- L2: Add weighted loss penalizing EOL errors — ~15 lines

Phase 2: Structural Improvements (Expected: Reduce MAE by 10-20 more hours)

These changes require more code but address fundamental architecture limitations:

- A1: Replace unidirectional GRU with BiGRU — ~5 lines in model.py
- A2: Add temporal attention pooling over all timesteps — ~15 lines in model.py
- A4: Include firmness as 3rd environmental feature — ~10 lines across model.py, dataset.py
- L3: Add binary EOL classification head (multi-task learning) — ~30 lines
- D1: Assign per-fruit EOL timestamps (requires relabeling data) — data work
- E1: Ensemble predictions from all 4 trained models — inference script only
- R4: Implement MixUp augmentation at batch level — ~20 lines in train.py

Phase 3: Advanced (Expected: Reduce MAE to 15-25 hours)

These changes require significant effort but could dramatically improve accuracy:

- D2: Acquire more fruit data (target: 20+ fruits) — most impactful single change
- A5: Upgrade to EfficientNet-B2 or ViT backbone (only after more data)
- E2: Self-supervised pretraining on fruit images (SimCLR/BYOL/MAE)
- E3: Multi-task learning with spoilage stage classification
- E5: Survival analysis formulation (DeepSurv / Cox PH)
- A3: Multi-layer RNN (2-3 layers) with proper inter-layer dropout

STRAWBERRY RUL PREDICTION — ROOT CAUSE DIAGNOSIS & IMPROVEMENT MAP

Current Performance: MAE 43-54h | RMSE 50-62h | Test: 1 fruit (F05) | Data: 6 strawberries

1. EOL FAILURE

Model predicts ~90h when fruit is spoiled (Actual RUL=0). RUL values clipped to >=0.

2. SEVERE OVERFITTING

Val loss INCREASES after epoch 3-4. 2614 train sequences from 4 fruits. 4.76M parameters.

3. SYSTEMATIC BIAS

Under-predicts RUL for fresh fruit. Over-predicts for spoiled fruit. Output range compressed.

4. LIMITED DATA

Only 6 fruits total. No augmentation. All share SAME EOL timestamp.

5. TRAINING GAPS

No early stopping, no LR schedule, no data augmentation. batch=4. Only RMSE loss.

6. ARCHITECTURE GAPS

Single RNN layer, last-timestep only. Only 2 env features. Crude normalization.

IMPROVEMENT DIRECTIONS

[A1] Allow negative RUL for post-EOL frames

[A2] Per-fruit EOL timestamps (not shared)

[A3] Huber / MSLE / Quantile loss

[A4] Classification head: spoiled yes/no

[B1] Data augmentation (rotate, color jitter, crop)

[B2] EarlyStopping + ReduceLROnPlateau

[B3] Weight decay / L2 regularization

[B4] Dropout increase: 0.2 -> 0.4-0.5

[C1] Multi-layer BiGRU / BiLSTM

[C2] Attention pooling (not just last timestep)

[C3] Include firmness as 3rd env feature

[C4] Proper z-score normalization

[D1] MixUp / CutMix augmentation

[D2] Ensemble all 4 model variants

[D3] Self-supervised pretraining on fruit

[D4] Larger EfficientNet (B1/B2) or ViT

[D5] Multi-task: RUL + spoilage stage + days

NOTE: These are suggestions for analysis — implementation should be prioritized based on feasibility and expected impact.

10. IMPLEMENTATION GUIDELINES

10.1 Code Changes: Data Augmentation (Phase 1 — R1)

Add the following transforms to `StrawberrySequenceDataset.__getitem__` in `dataset.py`. Apply augmentation ONLY during training, not validation/testing:

```
# In dataset.py __init__, add:
self.train_transform = transforms.Compose([
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.05),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomResizedCrop(224, scale=(0.9, 1.0)),
])

# In __getitem__, apply conditionally:
if self.training:
    img = self.train_transform(img)
img = self.transform(img) # existing normalization
```

10.2 Code Changes: Early Stopping (Phase 1 — R2)

Add early stopping logic to the training loop in `train.py`:

```
# Before training loop:
patience = 5
patience_counter = 0
best_val_loss = float("inf")
scheduler = ReduceLROnPlateau(optimizer, factor=0.5, patience=3)

# In epoch loop, after validation:
scheduler.step(val_loss)
if val_loss < best_val_loss - 0.5: # min_delta
    best_val_loss = val_loss
    patience_counter = 0
    torch.save(model.state_dict(), "best_model.pth")
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f"Early stopping at epoch {epoch+1}")
        break
```

10.3 Code Changes: Weighted EOL Loss (Phase 1 — L2)

Add per-sample loss weighting to prioritize accuracy near EOL:

```
# In train.py training loop, after computing loss:
# Compute per-sample weight based on actual RUL
actual_rul = ruls.squeeze() # (batch_size,)
eol_weight = 1.0 + 2.0 * torch.exp(-actual_rul / 20.0)
# weight=3.0 when RUL=0, weight~1.0 when RUL=200
weighted_loss = (loss_per_sample * eol_weight).mean()
weighted_loss.backward()
```

10.4 Code Changes: BiGRU + Temporal Attention (Phase 2 — A1, A2)

Replace the GRU with a bidirectional variant and add temporal attention:

```
# In model.py __init__:
self.gru = nn.GRU(
    input_size=self.rnn_input_size,
    hidden_size=rnn_hidden_size // 2, # half for bidirectional
    num_layers=num_layers,
    batch_first=True,
    bidirectional=True,
    dropout=dropout if num_layers > 1 else 0,
)

# Temporal attention weights
self.temporal_attn = nn.Linear(rnn_hidden_size, 1)

# In forward(), after GRU:
gru_out, _ = self.gru(fused_features) # (B, S, hidden)
attn_scores = self.temporal_attn(gru_out).squeeze(-1) # (B, S)
attn_weights = torch.softmax(attn_scores, dim=1) # (B, S)
context = torch.sum(gru_out * attn_weights.unsqueeze(-1), dim=1)
# context replaces gru_out[:, -1, :]
rul = self.regressor(context)
```

10.5 Code Changes: Multi-Task Head (Phase 2 — L3)

Add a binary classification head for EOL detection:

```
# In model.py __init__:
self.eol_classifier = nn.Sequential(
    nn.Linear(rnn_hidden_size, 32),
    nn.ReLU(),
    nn.Linear(32, 1), # binary: spoiled or not
)

# In forward(), return both:
rul = self.regressor(context)
eol_logit = self.eol_classifier(context)
return rul, eol_logit

# In train.py, compute combined loss:
rul_pred, eol_logit = model(images, envs)
loss_reg = criterion_reg(rul_pred, ruls)
eol_target = (ruls <= 0).float().unsqueeze(1)
loss_cls = criterion_cls(eol_logit, eol_target)
loss = loss_reg + 0.1 * loss_cls
```

11. EXPECTED IMPROVEMENT TRAJECTORY

The following table estimates the expected performance improvement as each phase is implemented. These are conservative estimates based on typical improvements reported in similar computer vision regression tasks with small datasets.

Phase / Change	MAE (h)	RMSE (h)	R-squared	EOL Error (h)
Current (Model A)	43.1	49.9	0.61	~90
Phase 1: Quick Wins	28-33	35-40	0.70-0.75	~40-50
Phase 2: Structural	18-25	25-32	0.78-0.84	~15-25
Phase 3: Advanced	10-18	15-25	0.85-0.92	~5-10
TARGET (Production)	<12	<15	>0.90	<5

Key Assumptions

- Phase 1 improvements come primarily from reduced overfitting (early stopping, augmentation) and better optimization (Huber loss, EOL weighting).
- Phase 2 improvements come from the BiGRU providing bidirectional temporal context and the multi-task head providing explicit EOL signal.
- Phase 3 improvements come from more data (most impactful), better backbone, and advanced pretraining.
- The EOL Error is expected to improve the most dramatically because the current architecture has NO mechanism to distinguish EOL vs non-EOL beyond the continuous RUL regression — the multi-task classification head directly addresses this.

12. MONITORING & VALIDATION PROTOCOL

To ensure improvements are genuine and not due to chance, follow this validation protocol for each change:

- 1. ISOLATE CHANGES: Test each improvement independently before combining. This identifies which changes actually help vs which are neutral or harmful.
- 2. FIXED SEEDS: Set random seeds (torch.manual_seed, np.random.seed) for reproducibility. Run each experiment 3 times with different seeds and report mean ± std.
- 3. TRACK METRICS: For each experiment, record MAE, RMSE, R-squared, and most importantly, per-RUL-range errors (0-24h, 24-72h, 72-120h, 120-168h, 168-260h).
- 4. VISUAL INSPECTION: Plot predicted vs actual for each experiment. A lower MAE is meaningless if the EOL predictions haven't improved.
- 5. ABLATION STUDIES: Once all improvements are combined, run ablation studies removing one component at a time to measure its marginal contribution.
- 6. TEST SET INTEGRITY: NEVER use F05 (test fruit) for hyperparameter tuning. Use only F06 (validation fruit) for all tuning decisions. Reserve F05 for final evaluation only.

IMPORTANT: Do NOT tune hyperparameters using test set (F05) performance. The test set must remain completely untouched until final evaluation. All tuning decisions should use validation set (F06) only. Violating this rule produces overly optimistic results that won't generalize to new fruit.

13. CONCLUSION

The Strawberry RUL prediction project has a solid foundation: a well-structured data pipeline, a sound architectural design (CNN + Attention + RNN), proper fruit-ID-safe splits, and thorough documentation. The current MAE of 43.1 hours on the test fruit represents a functional prototype.

However, the model suffers from three critical issues that make it unsuitable for practical deployment:

- It cannot recognize spoiled fruit (predicts ~90h RUL when the fruit is already at EOL).
- It severely overfits the 4 training fruits (generalization gap >13 hours).
- It compresses its output range, failing to distinguish fresh fruit from aging fruit.

The 21 improvement suggestions in this report are organized into three phases, prioritized by impact-to-effort ratio. Phase 1 (Quick Wins) can be implemented in 2-3 days and is expected to reduce MAE by 25-35%. Phase 2 addresses fundamental architecture limitations and is expected to cut MAE roughly in half. Phase 3 requires more data and effort but could bring the model to production-ready accuracy.

Top 5 Highest-Priority Actions

- 1. Add data augmentation + early stopping (Phase 1, R1+R2) — prevents overfitting immediately
- 2. Implement weighted EOL loss (Phase 1, L2) — directly addresses the EOL failure problem
- 3. Remove RUL clipping to allow negative values (Phase 1, EOL Solution 1) — teaches model about post-EOL state
- 4. Add BiGRU + temporal attention (Phase 2, A1+A2) — better temporal modeling
- 5. Add multi-task EOL classification head (Phase 2, L3) — explicit EOL detection signal

The single most impactful change — acquiring more fruit data — is listed in Phase 3 due to the effort required, but if resources permit, it should be prioritized above all other changes. Deep learning models are fundamentally data-hungry; no amount of architectural cleverness can fully compensate for having only 4 training fruits.

— End of Report —